



Programación 2

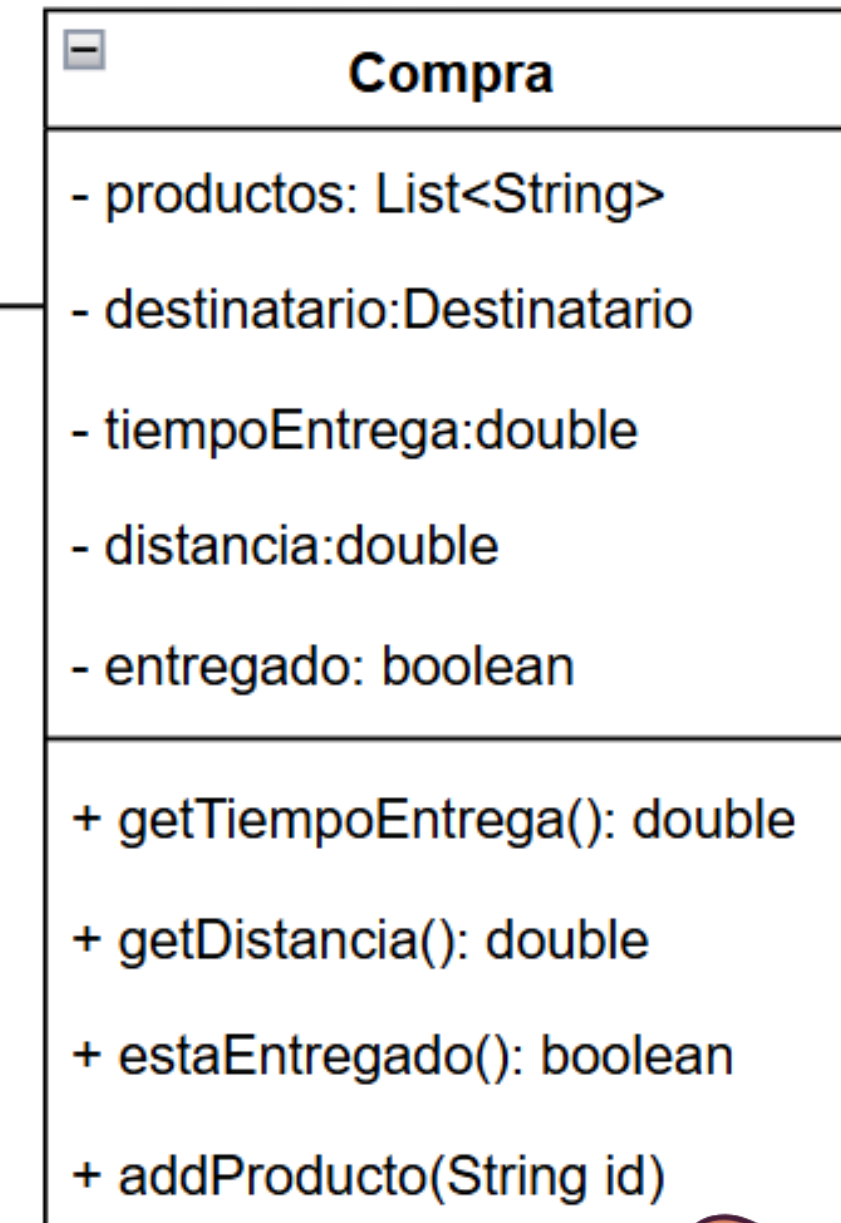
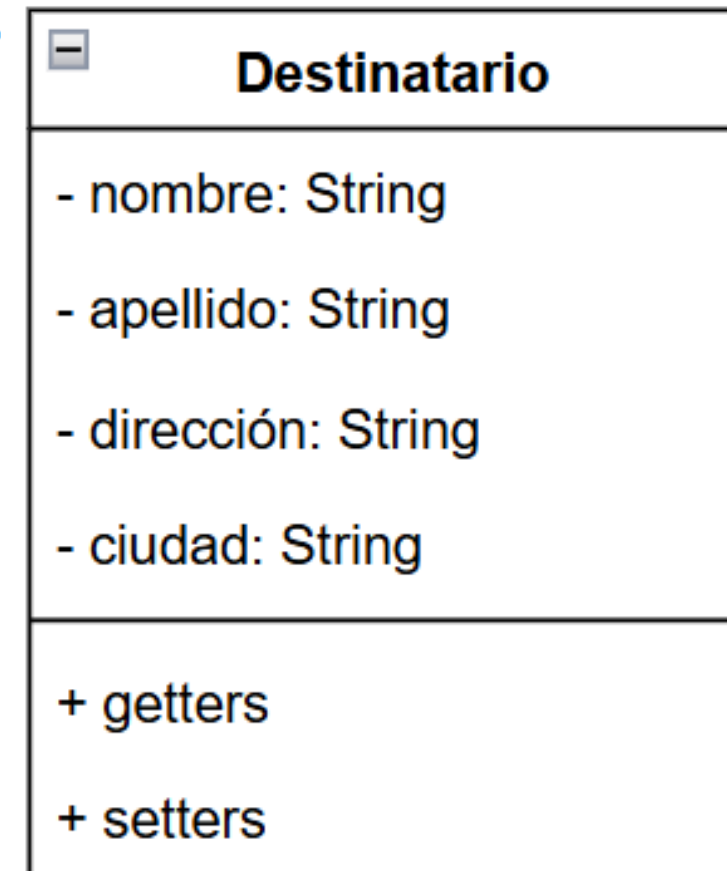
TUDAI

Recuperatorio 2025

Compras

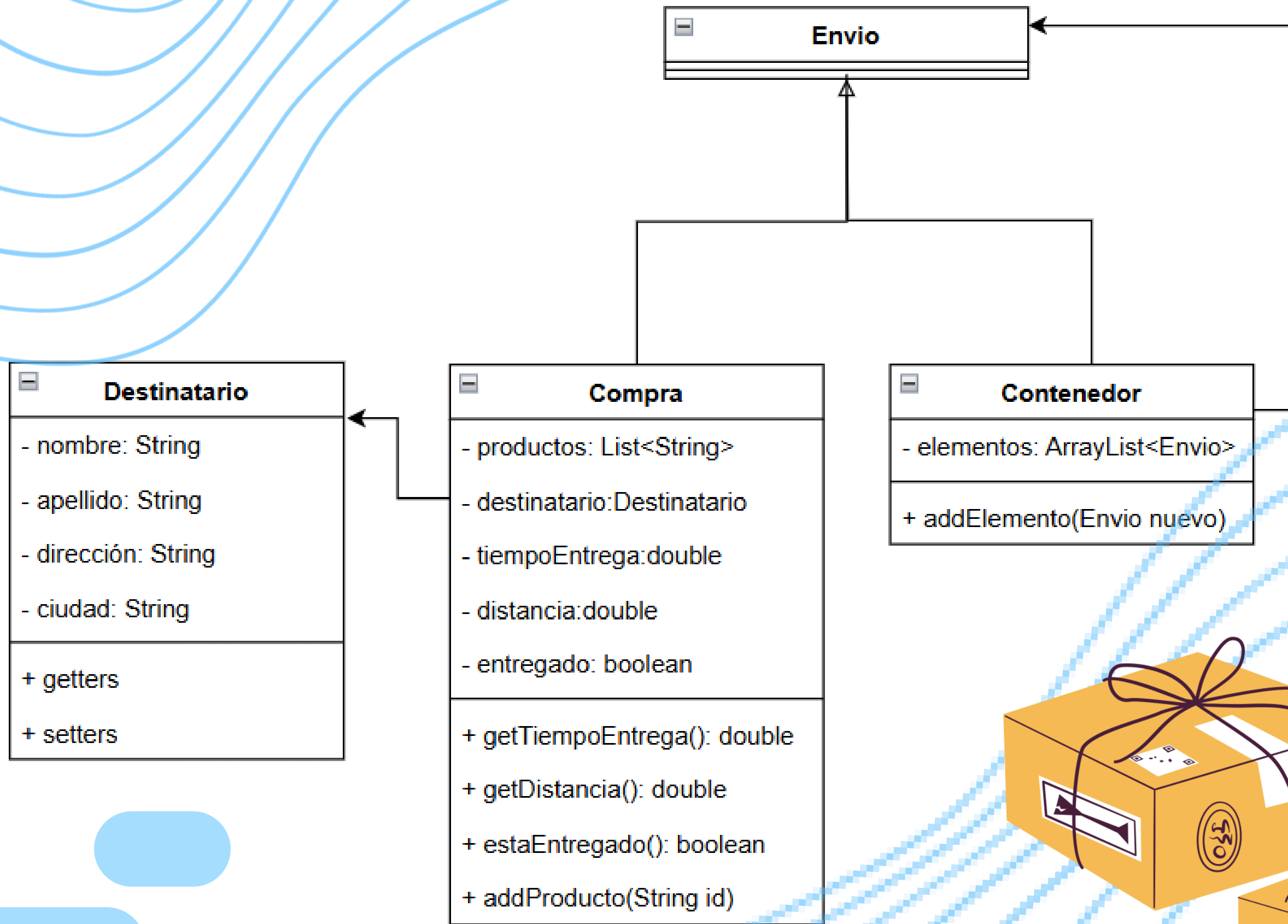
Una empresa de transporte gestiona entregas a domicilio de compras online. Cada compra consiste de uno o más identificadores de productos, un destinatario (con nombre, apellido, dirección y ciudad), si ya se entregó o no, tiempo estimado de entrega (en horas), y la distancia desde la central de despachos (en kilómetros).





Para optimizar la logística, las compras se agrupan en diferentes contenedores. A su vez, un contenedor, además de compras, puede tener otros contenedores.

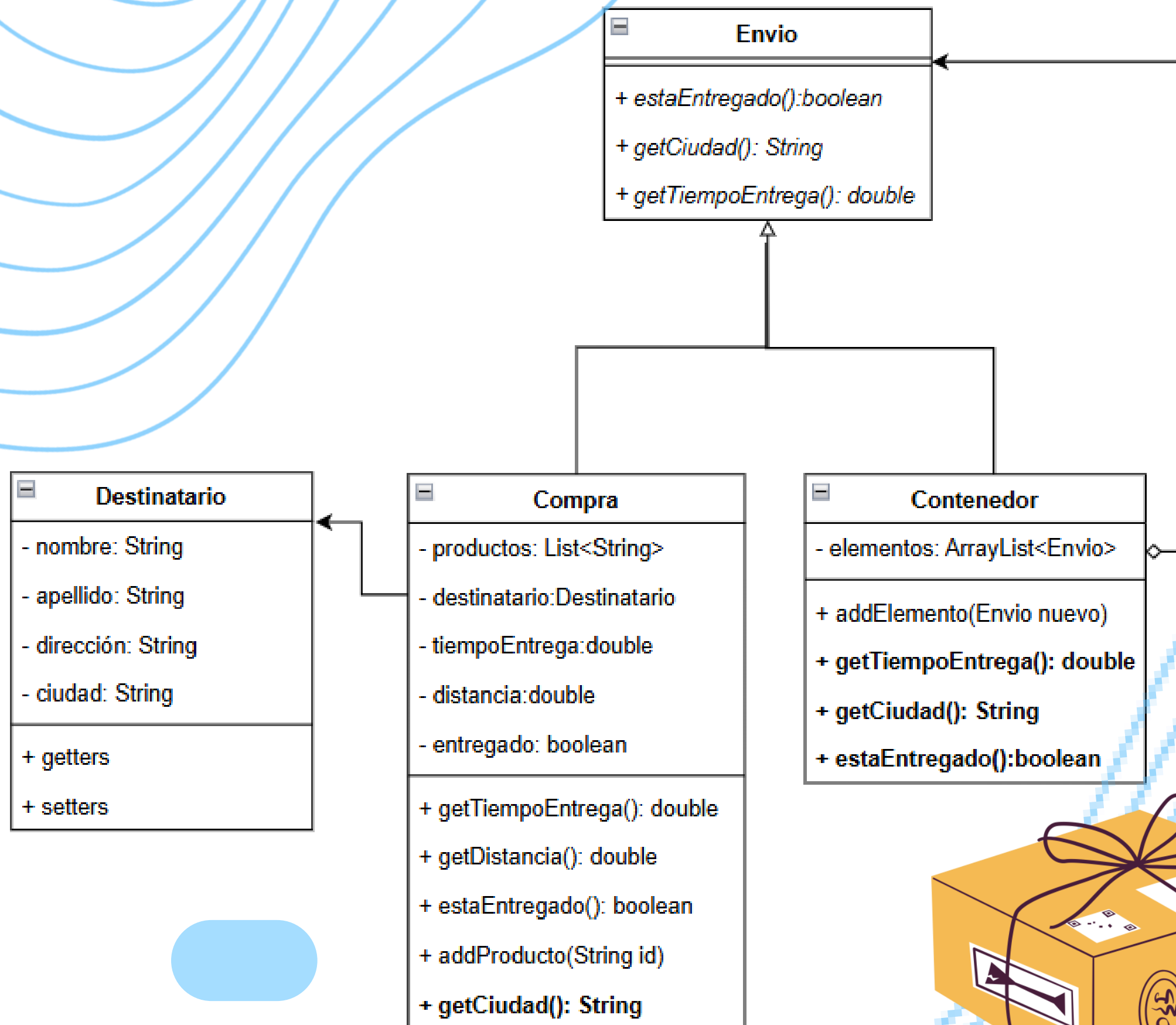






Un contenedor se considera **entregado** si *todos* sus elementos están entregados. La **ciudad de destino** de un contenedor queda determinada por la ciudad del *primer elemento* que contiene. Por otra parte, el **tiempo estimado** de un contenedor queda determinado por el *máximo* tiempo estimado de sus elementos.





@Override

```
public boolean estaEntregado() {  
    if (this.elementos.isEmpty()) {  
        return false;  
    }  
    for (Envio elemento : this.elementos) {  
        if (!elemento.estaEntregado()) {  
            return false;  
        }  
    }  
    return true;  
}
```

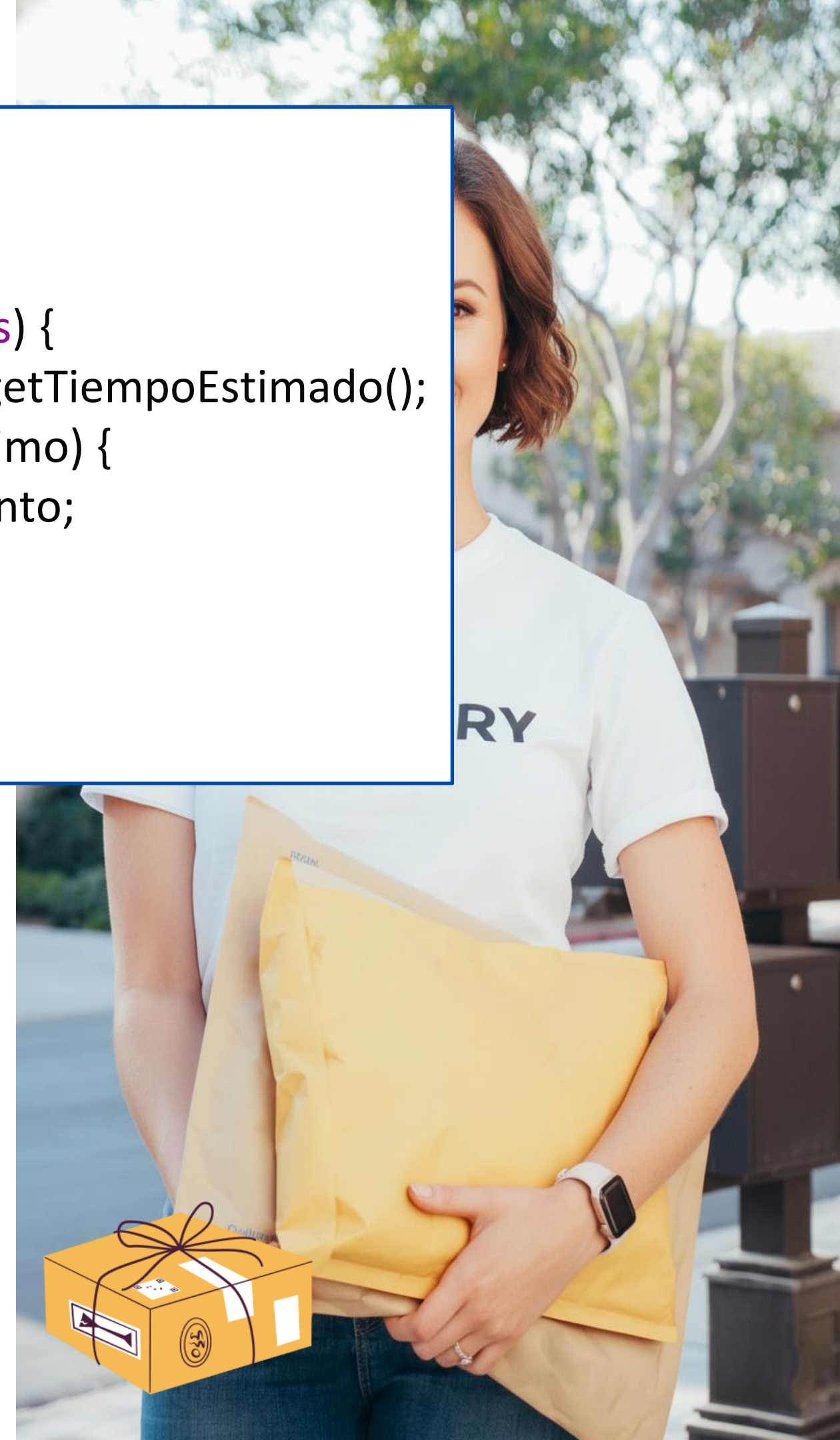
@Override

```
public int getTiempoEstimado() {  
    int tiempoMaximo = 0;  
    for (Envio elemento : this.elementos) {  
        int tiempoElemento = elemento.getTiempoEstimado();  
        if (tiempoElemento > tiempoMaximo) {  
            tiempoMaximo = tiempoElemento;  
        }  
    }  
    return tiempoMaximo;  
}
```

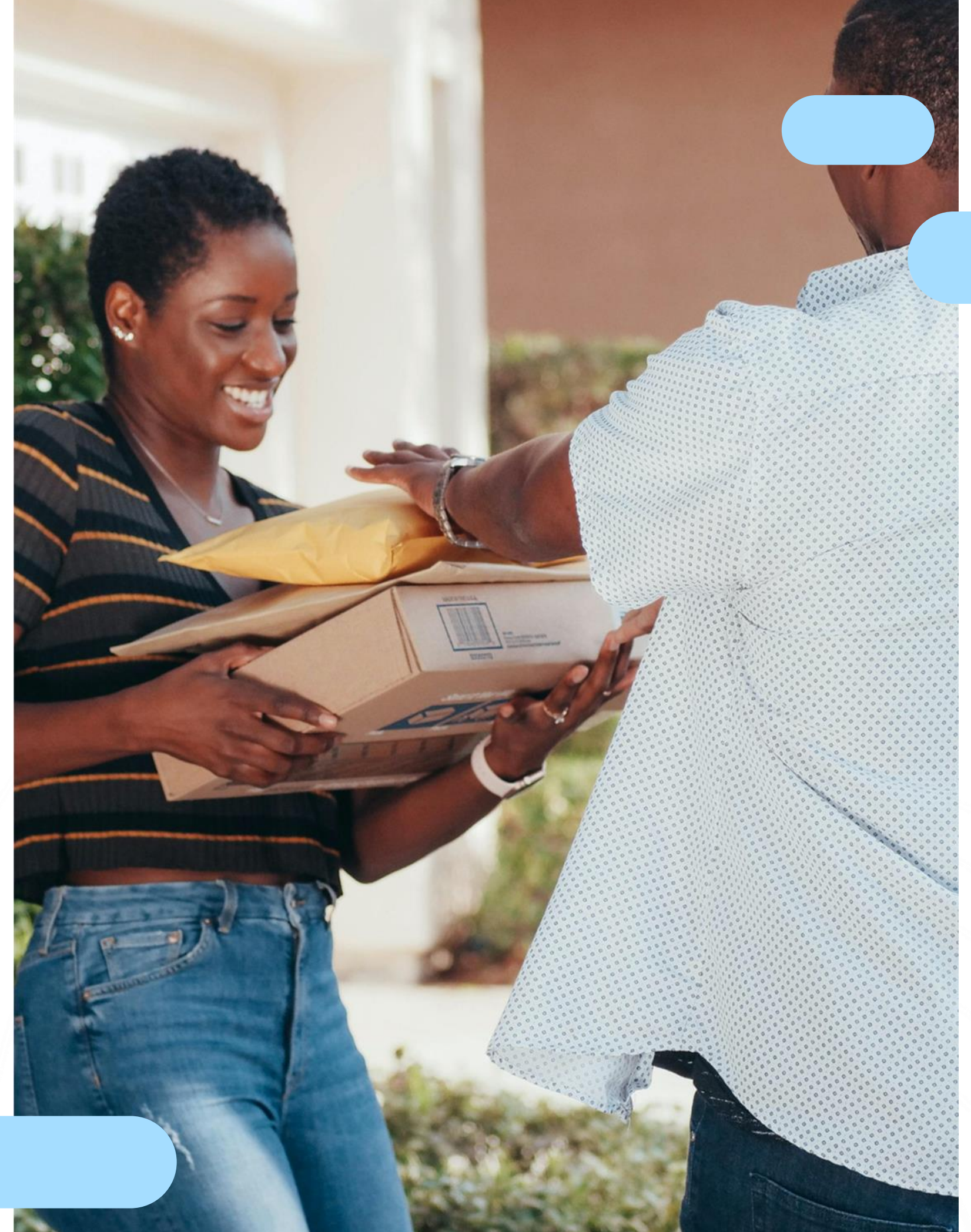
@Override

```
public String getCiudadDestino() {  
    if (this.elementos.isEmpty())  
        return null;  
    else  
        return this.elementos.get(0).getCiudadDestino();  
}
```

Contenedor

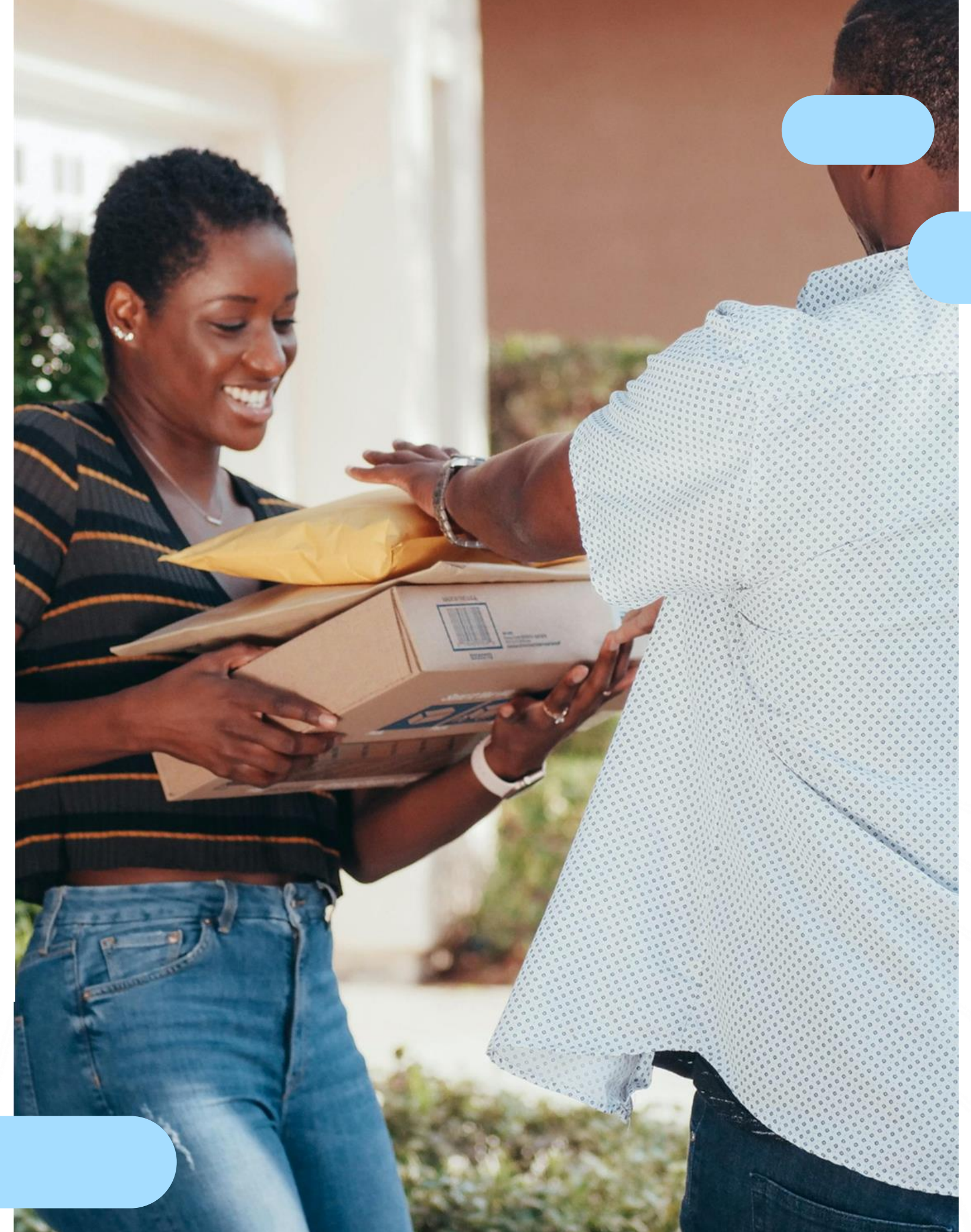


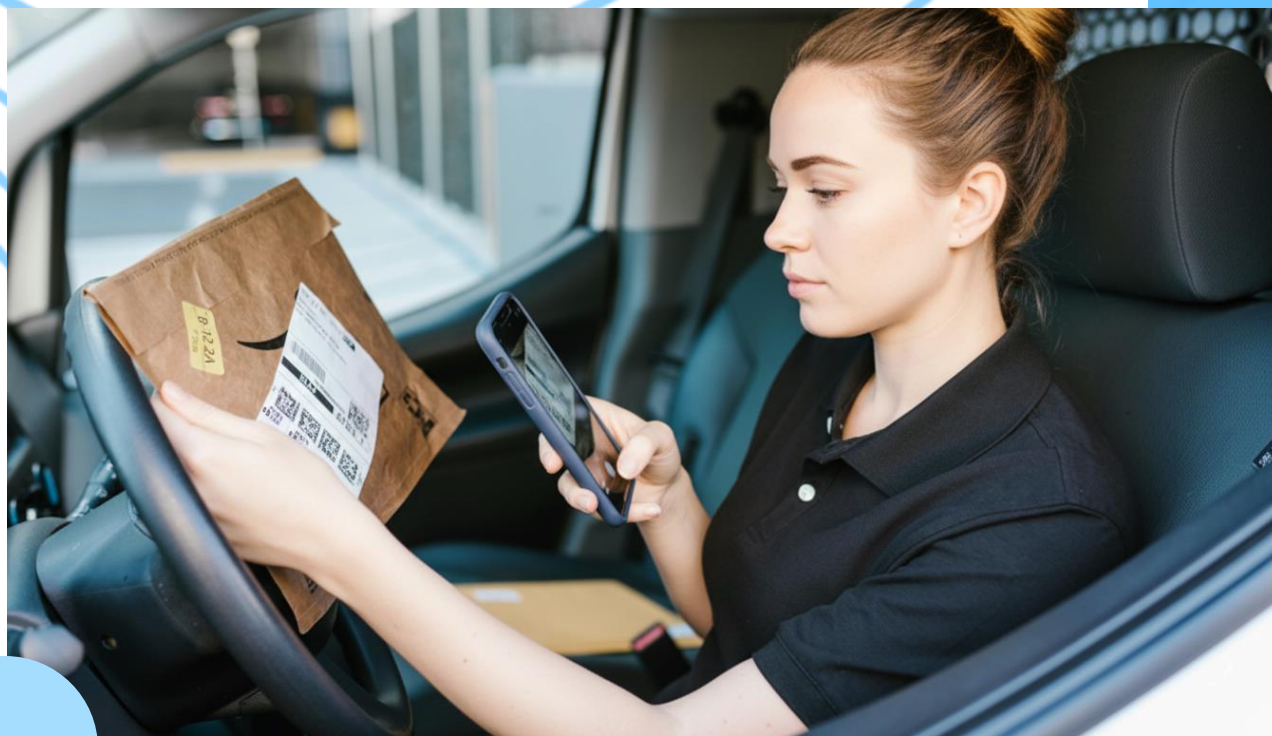
Cuando se agrega un elemento a un contenedor, **siempre se chequea** que la ciudad del destinatario del nuevo elemento sea la misma que la del contenedor al que se desea agregar (si no lo es, no se agrega; si no está determinada aún en el contenedor, se agrega).



Contenedor

```
public void addElemento(Envio elemento) {  
    String destino = this.getCiudadDestino();  
    if (destino == null ||  
        destino.equals(elemento.getCiudadDestino())) {  
        this.elementos.add(elemento);  
    }  
}
```





El usuario debe poder consultar a un contenedor cuál es la **siguiente compra** a repartir, independientemente del nivel en el que se encuentre. Para esto, puede optar por diferentes **estrategias** de distribución, las cuales determinan el orden en el que se repartirán las compras. La siguiente compra a repartir es la primera acorde a este orden y que no ha sido entregada aún.

Existen actualmente tres estrategias que puede usar el usuario al consultar por la siguiente compra a repartir, pero se deben poder agregar nuevas estrategias a futuro.

Estas estrategias son:



Rápida

prioriza el tiempo estimado de envío (se entregan primero las compras con menor tiempo estimado de entrega)



Ecológica

prioriza el bajo consumo de combustible, entregando primero las compras que tengan una menor distancia

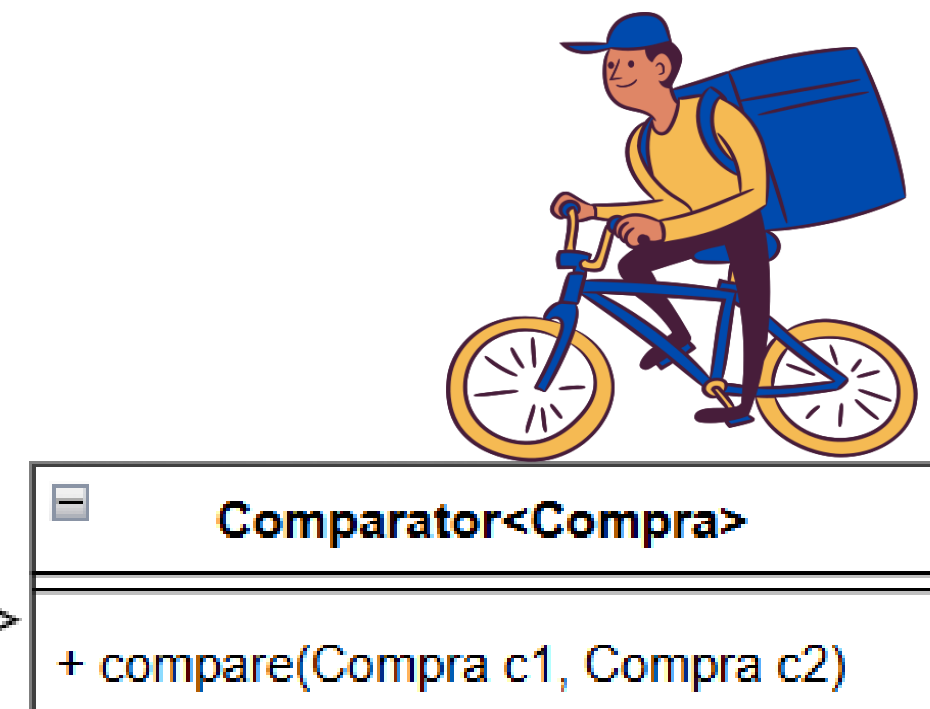
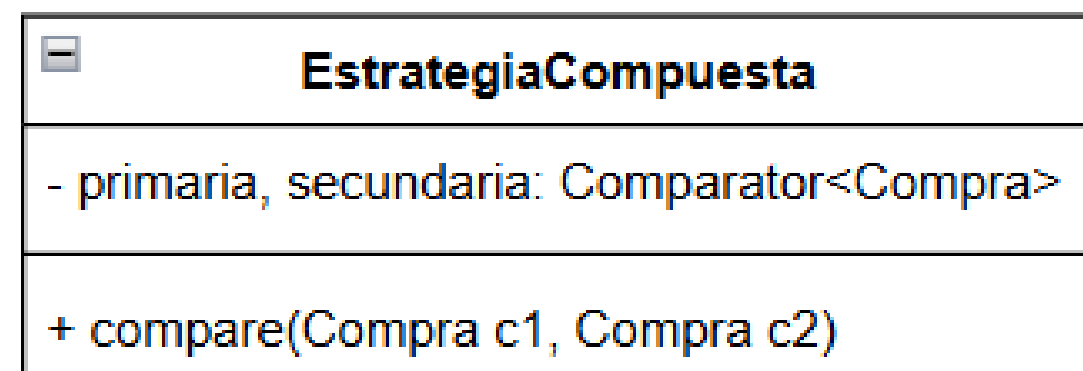
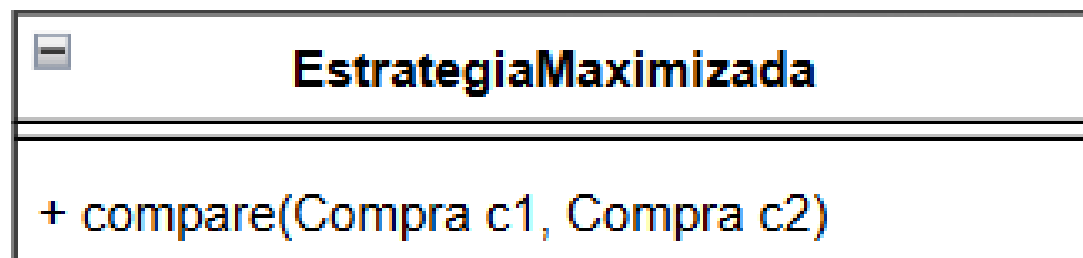
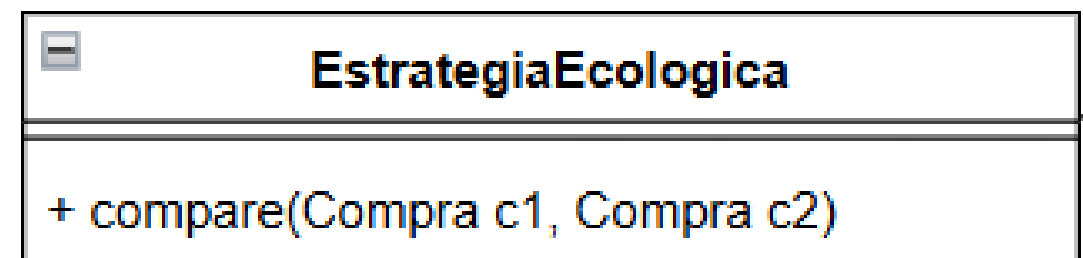
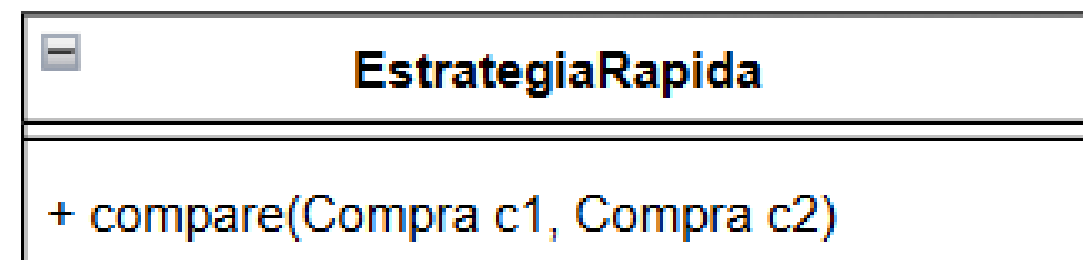


Maximizada

prioriza las compras que tengan mayor cantidad de productos

También debe ser posible crear nuevas estrategias compuestas en las que, si dos compras “empatan”, se use otra estrategia para desempatar.

Estrategias → Comparator<Compra>



Estrategias ➡ Comparator<Compra>

```
public class EstrategiaRapida implements Comparator<Compra> {  
    @Override  
    public int compare(Compra o1, Compra o2) {  
        return Double.compare( o1.getTiempoEstimado(), o2.getTiempoEstimado());  
    }  
}
```



```
public class EstrategiaEcologica implements Comparator<Compra> {  
    @Override  
    public int compare(Compra o1, Compra o2) {  
        return Double.compare(o1.getKilometros(), o2.getKilometros());  
    }  
}
```

Estrategias ➡ Comparator<Compra>

```
public class EstrategiaMaximizada implements Comparator<Compra> {  
    @Override  
    public int compare(Compra o1, Compra o2) {  
        return o2.getCantidadProductos() - o1.getCantidadProductos();  
    }  
}
```



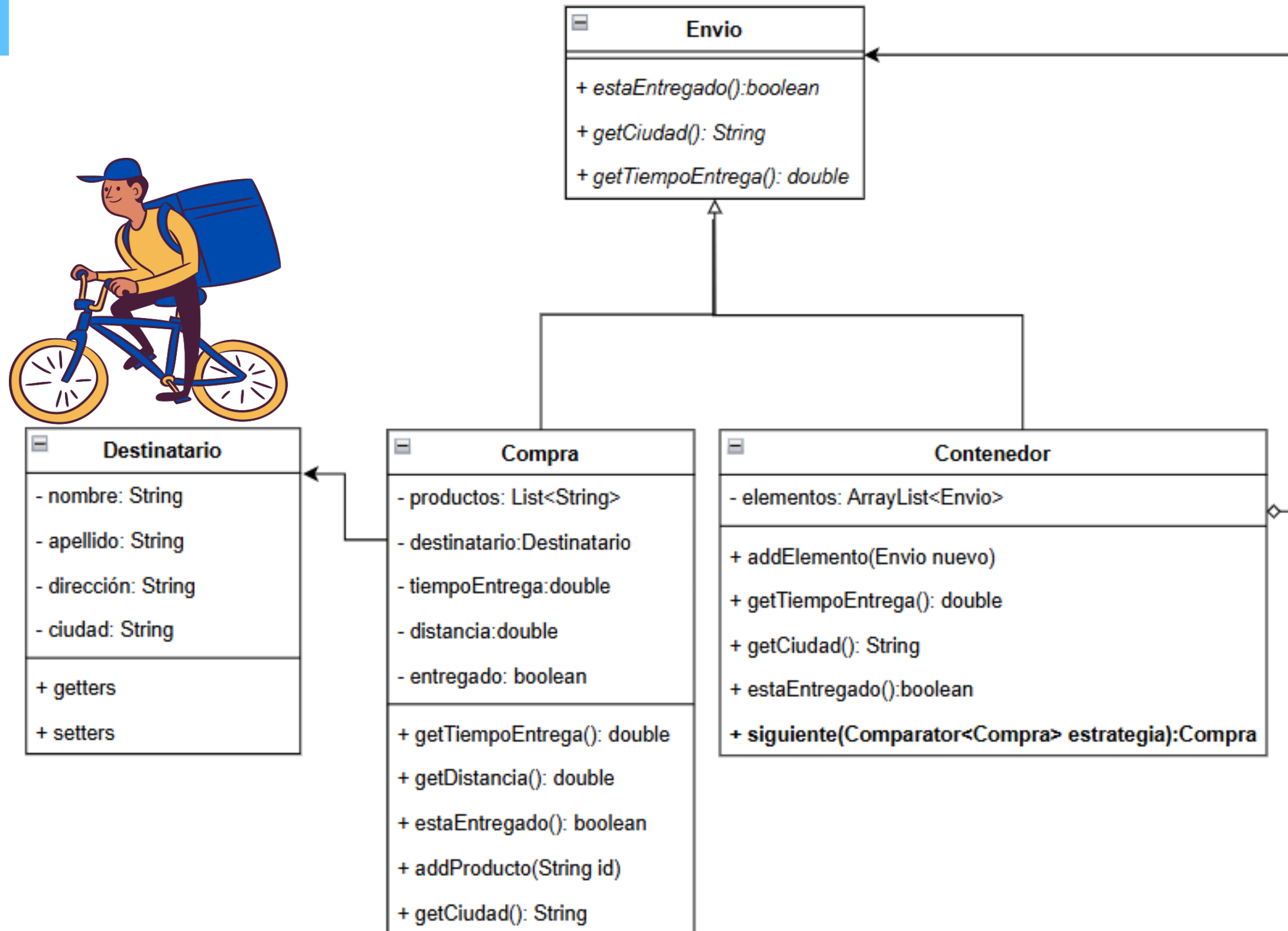
Implementar este método!

Estrategias → Comparator<Compra>

```
public class EstrategiaCompuesta implements Comparator<Compra> {  
    private Comparator<Compra> estrategiaPrincipal, estrategiaSecundaria;  
  
    public EstrategiaCompuesta(Comparator<Compra> estrategiaPrincipal, Comparator<Compra> estrategiaSecundaria) {  
        this.estrategiaPrincipal = estrategiaPrincipal;  
        this.estrategiaSecundaria = estrategiaSecundaria;  
    }  
  
    @Override  
    public int compare(Compra o1, Compra o2) {  
        int res = this.estrategiaPrincipal.compare(o1, o2);  
        if (res == 0) {  
            return this.estrategiaSecundaria.compare(o1, o2);  
        }  
        return res;  
    }  
}
```



¿Y cómo la uso?



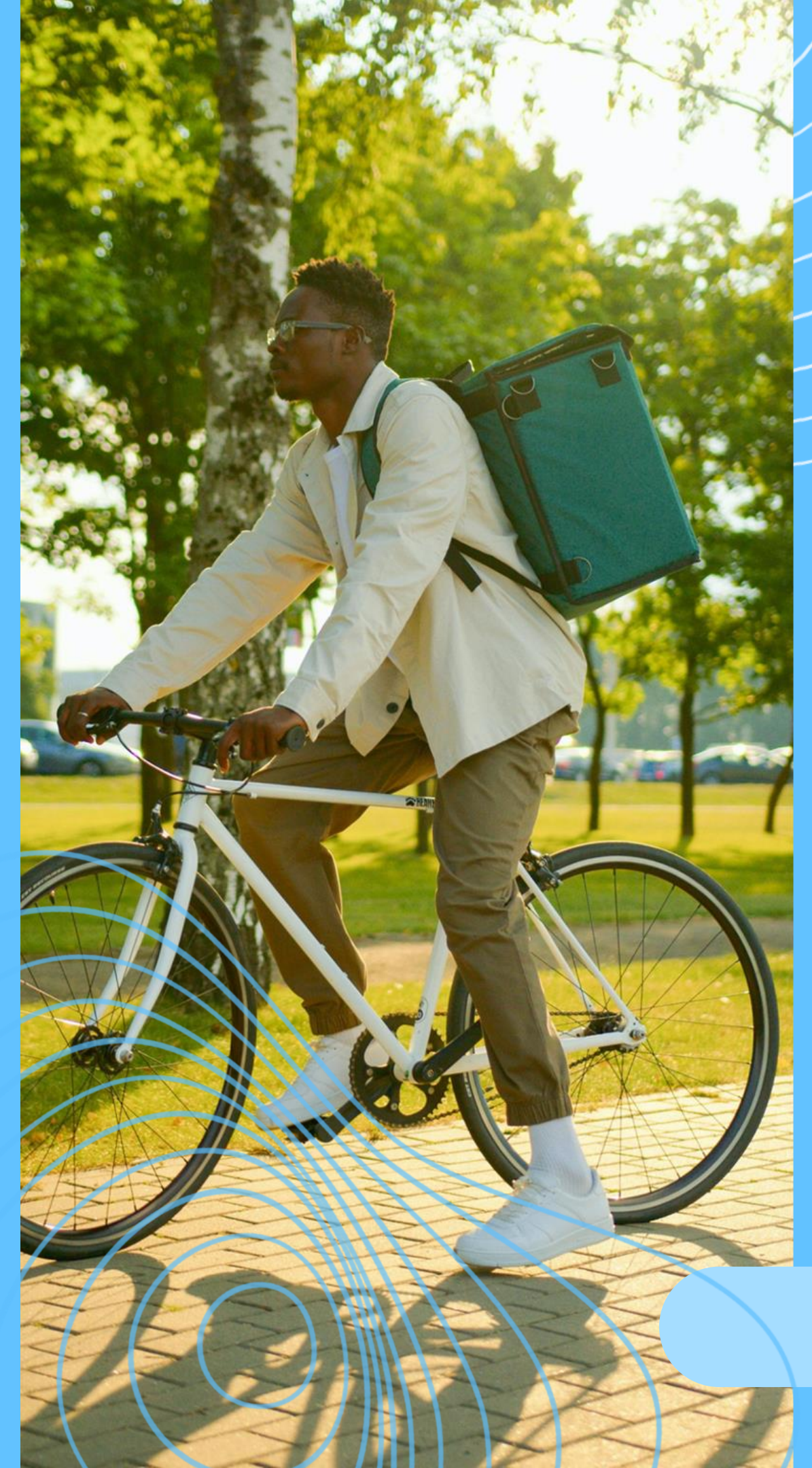
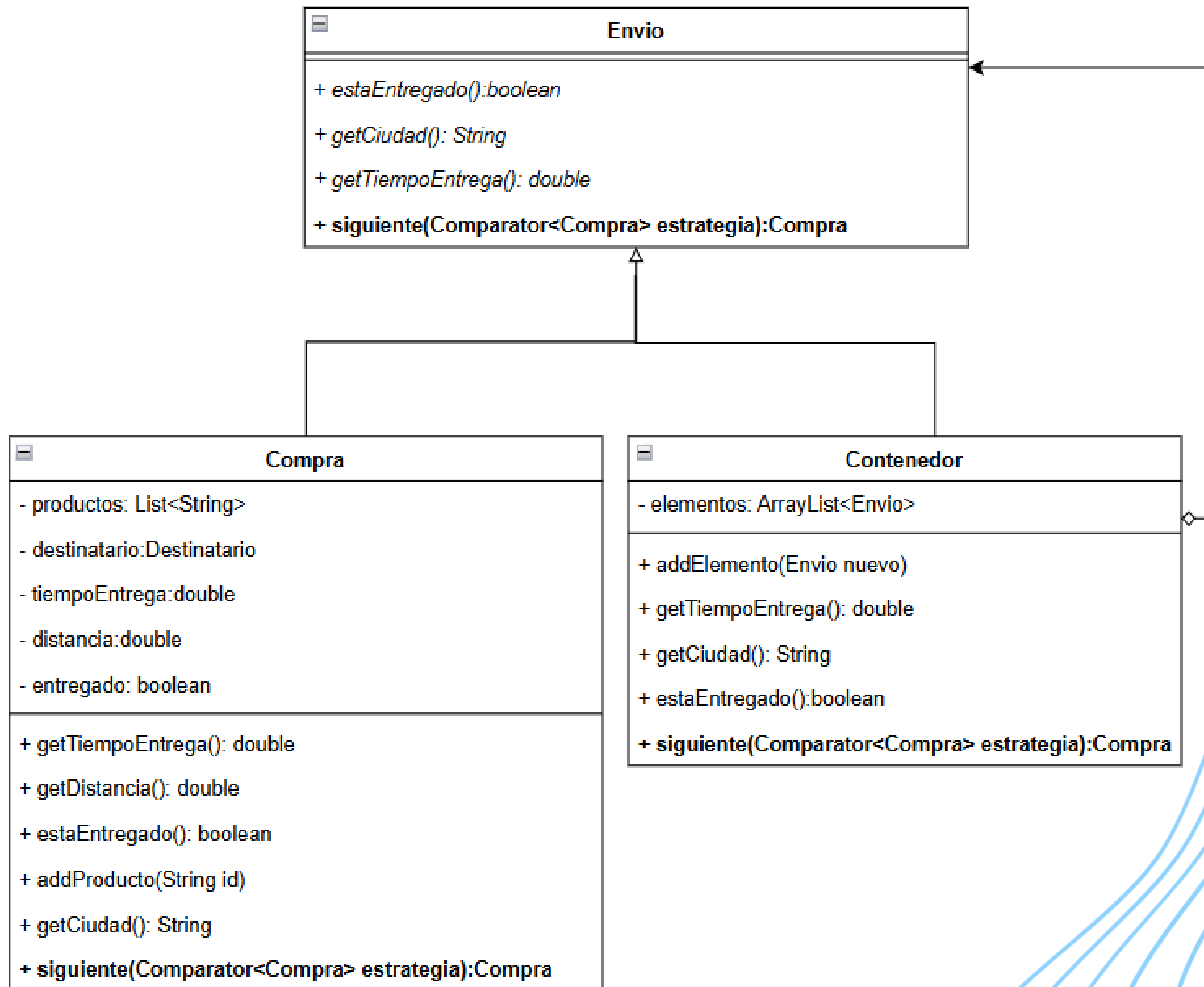
La solución que esperábamos...

```
public Compra siguiente(Comparator<Compra> estrategia) {  
    Compra siguienteReparto = null;  
    for (Envio elemento : this.elementos) {  
        Compra siguienteDelElemento = elemento.siguiente(estrategia);  
        if (siguienteReparto == null || (siguienteDelElemento != null &&  
            estrategia.compare(siguienteDelElemento, siguienteReparto) < 0)) {  
            siguienteReparto = siguienteDelElemento;  
        }  
    }  
    return siguienteReparto;  
}
```

Contenedor



Debemos abstraer el método



La solución que esperábamos...

@Override

```
public Compra siguiente(Comparator<Compra> estrategia) {  
    if ( !this.estaEntregado())  
        return this;  
    else return null;  
}
```

Compra

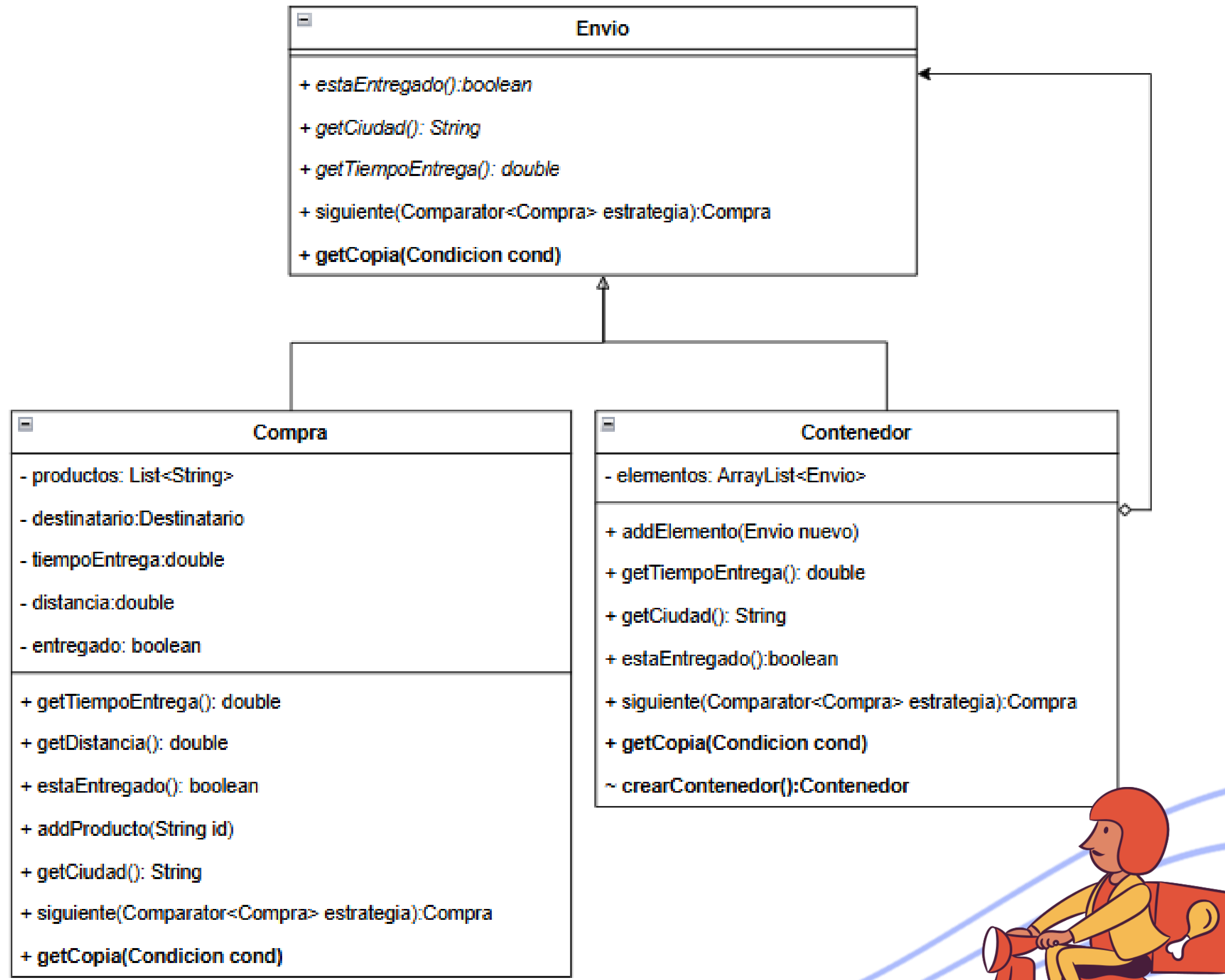




El sistema debe permitir crear una copia en profundidad de un contenedor dado (al modificar la copia no se modifica el original) es importante que se mantenga su estructura (la organización en niveles de contenedores), pero conservando solo las compras que posean ciertas características, por ejemplo:

- **Conservar sólo las compras con más de 5 productos**
- **Conservar sólo las compras cuya ciudad destino sea de Tandil**
- **Conservar sólo las compras que no han sido entregadas.**
- **Combinaciones lógicas de los ejemplos anteriores o de nuevas restricciones que se puedan aplicar sobre las compras.**

En el caso de los contenedores, si los mismos quedan sin elementos, los mismos NO deben incluirse en el resultado.



Compra

@Override

```
public Envio getCopia(Condicion condicion) {  
    if (condicion.cumple(this)) {  
        Compra copiaCompra = new Compra(this.destinatario,  
                                           this.tiempoEstimado,  
                                           this.kilometros);  
        copiaCompra.setEntregado(this.entregado);  
        for (String producto: productos)  
            copiaCompra.addProducto(producto);  
    }  
    return null;  
}
```

No Olvidarse de copiar TODO



Contenedor

@Override

```
public Envio getCopia(Condicion condicion) {  
    Contenedor copia = crearContenedor();  
    boolean copiar = false;  
    for (Envio elemento : this.elementos) {  
        Envio copiaElemento = elemento.getCopia(condicion);  
        if (copiaElemento != null) {  
            copia.addElemento(copiaElemento);  
            copiar = true;  
        }  
    }  
    if (copiar)  
        return copia;  
    else  
        return null;  
}
```

Esto es lo que evita copiar
contenedores sin
elementos

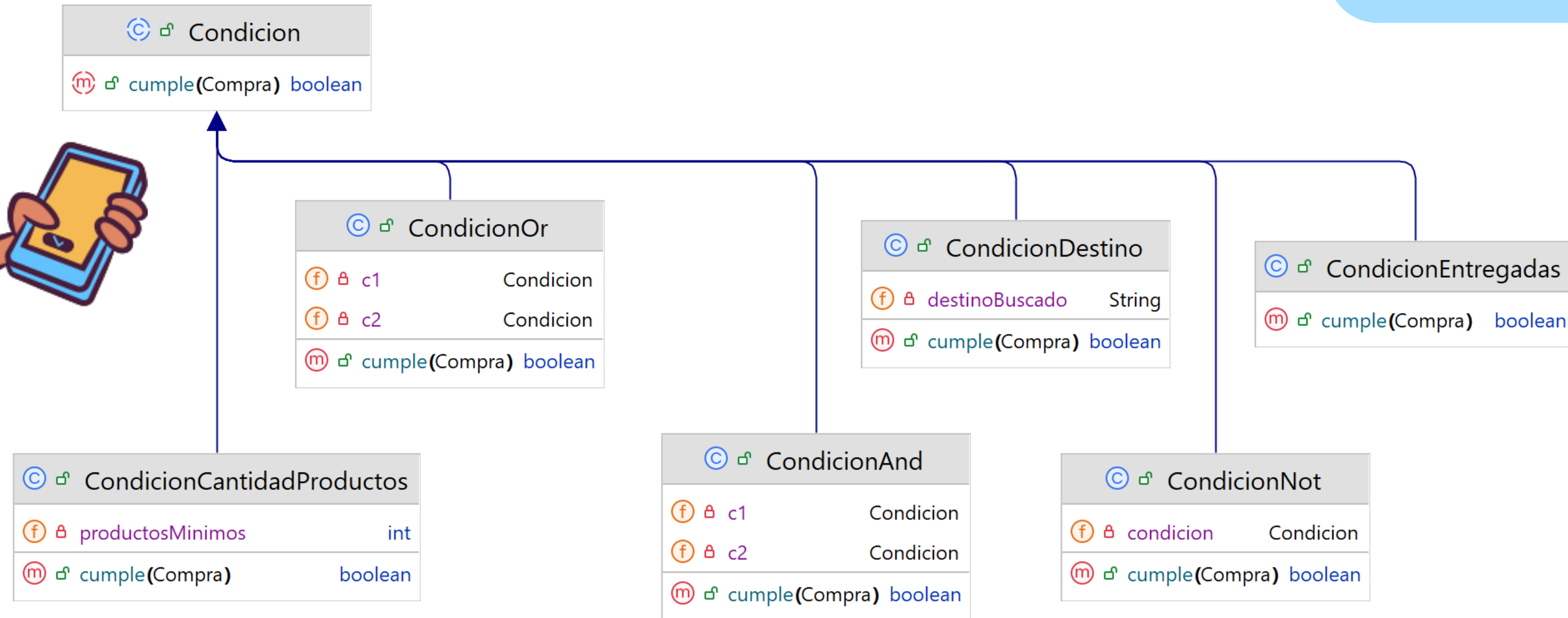
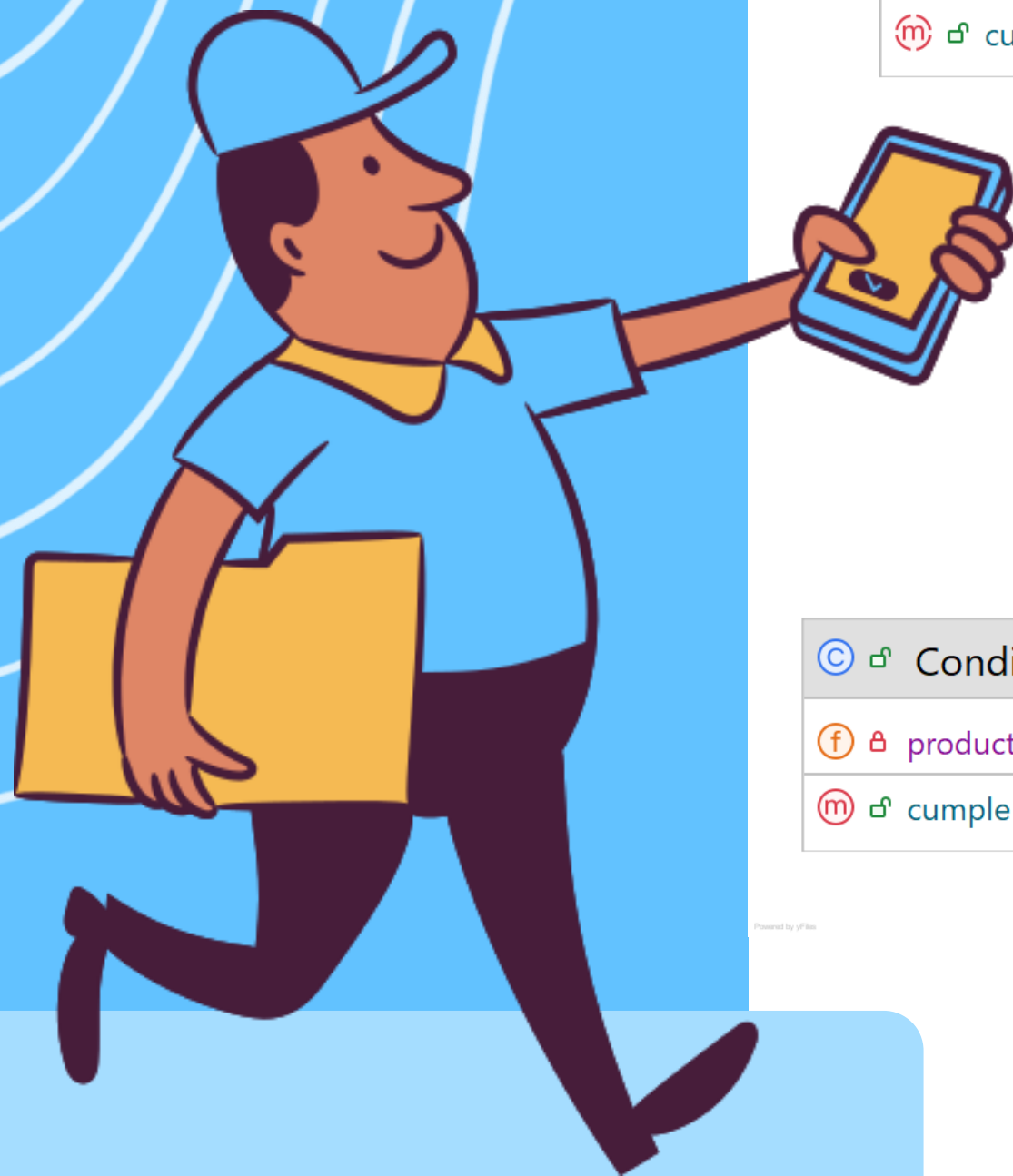


Contenedor

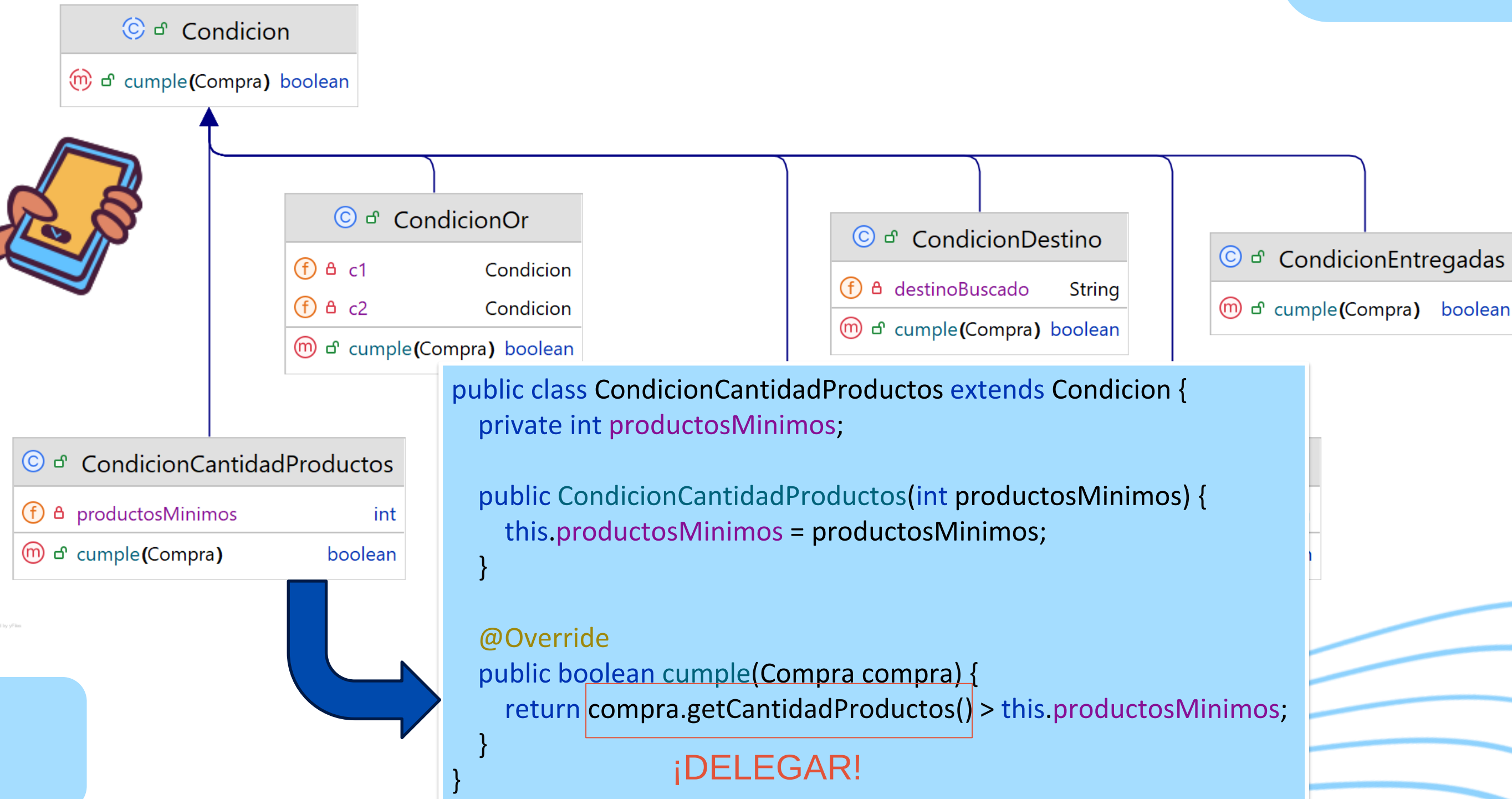
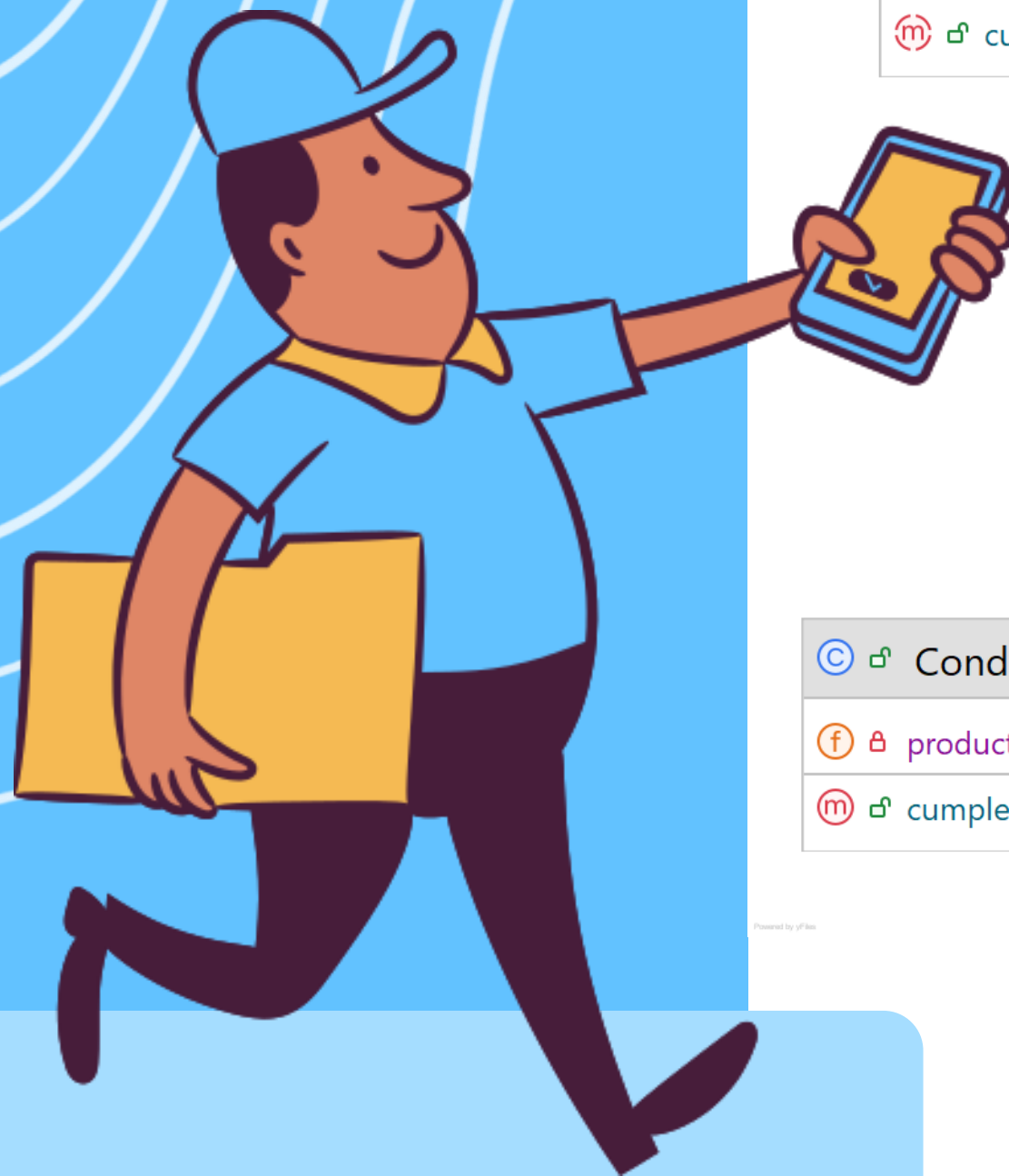
```
protected Contenedor crearContenedor() {  
    return new Contenedor();  
}
```



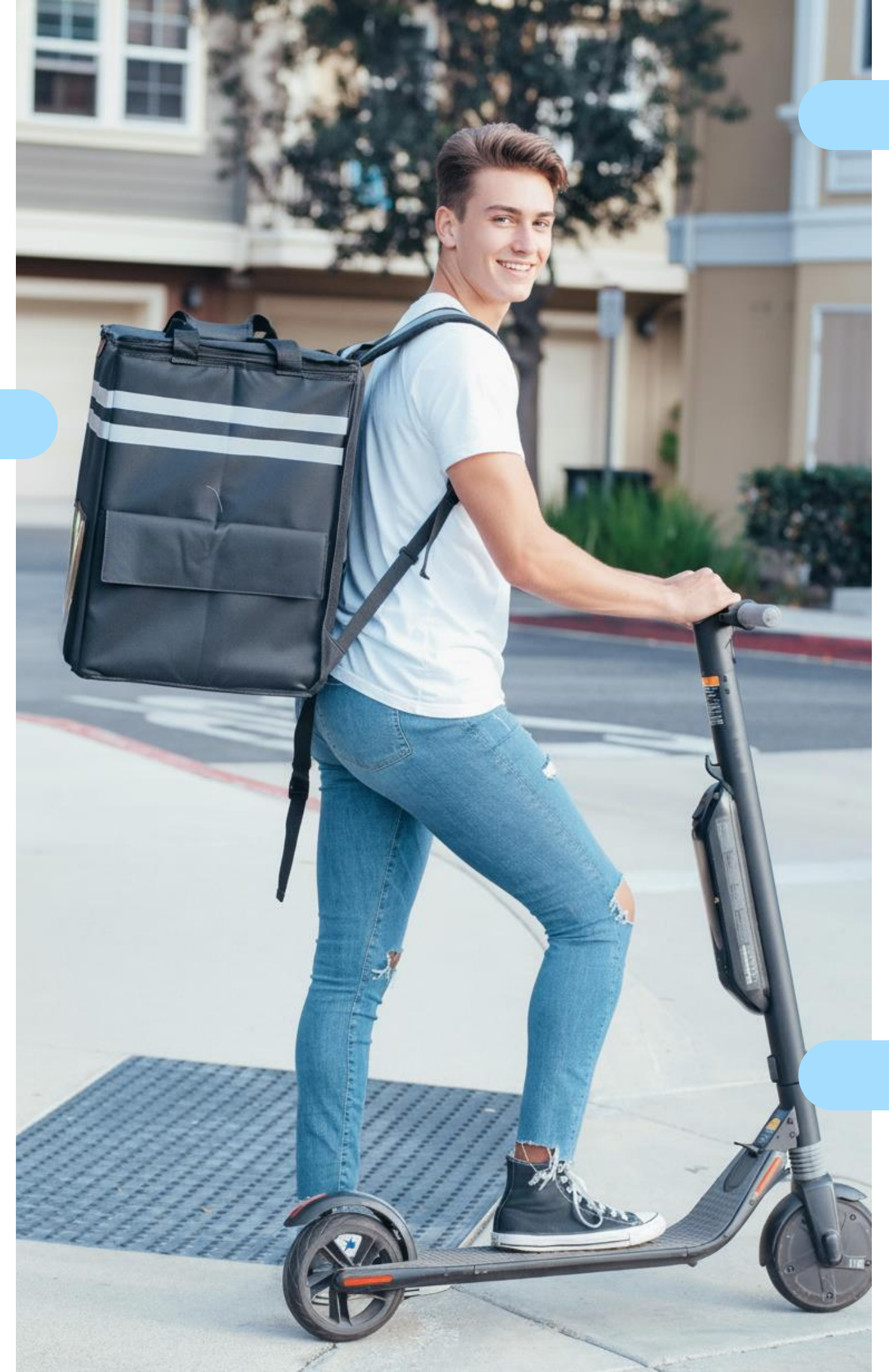
Las condiciones, a esta altura, ya están más que sabidas

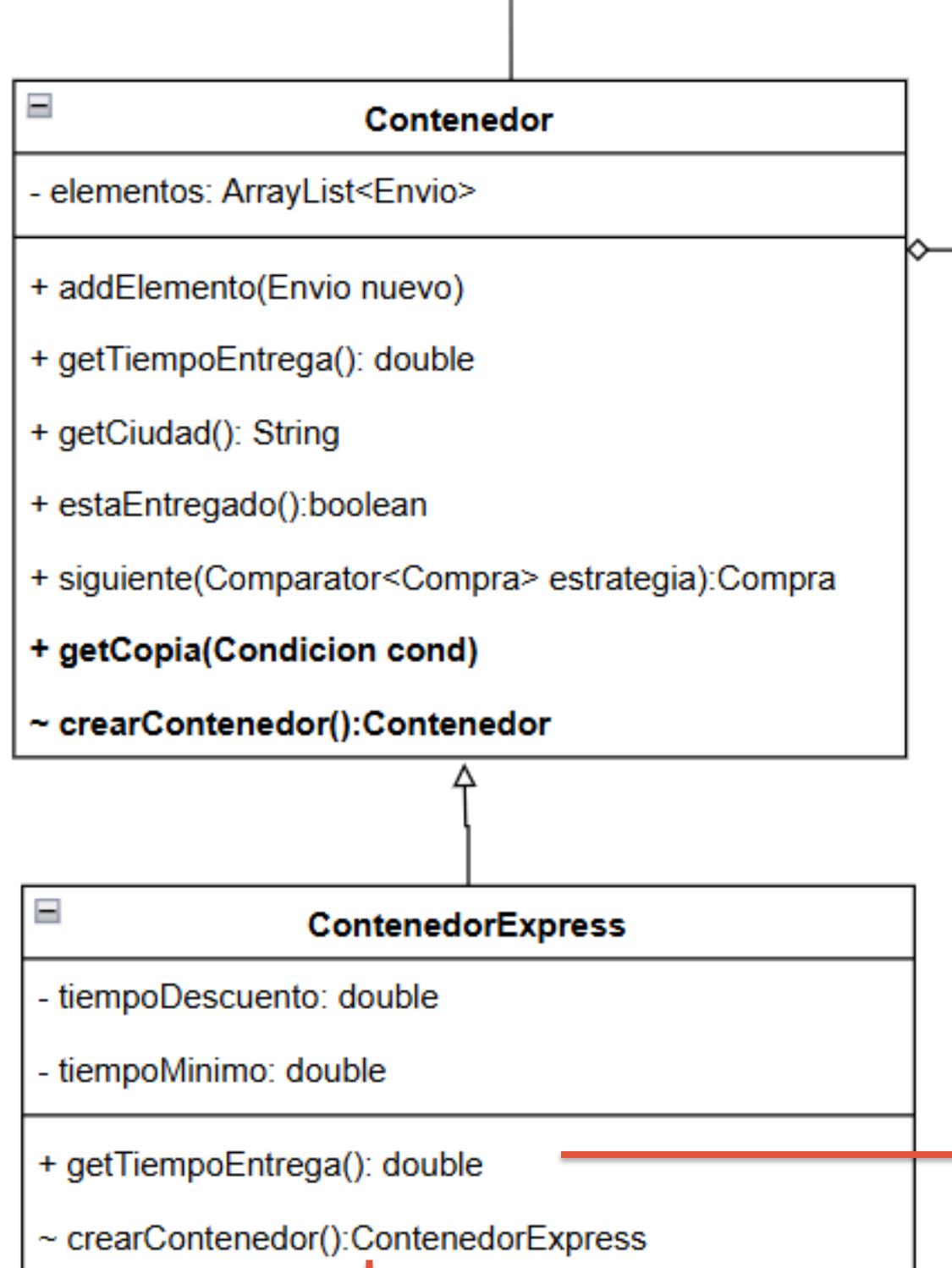


Veamos una, por ejemplo...



Además, la empresa quiere incorporar **Contenedores Express**, para los cuales el tiempo de entrega es de 5 horas menos que el máximo tiempo de sus elementos, pero nunca puede ser inferior a 1 hora.





@Override

```

protected Contenedor crearContenedor() {
    return new ContenedorExpress(this.tiempoDescuento,
                                this.tiempoMinimo);
}
  
```

@Override

```

public int getTiempoEstimado() {
    int tiempoEstimado = super.getTiempoEstimado() - tiempoDescuento;
    if (tiempoEstimado >= tiempoMinimo)
        return tiempoEstimado;
    else return tiempoMinimo;
}
  
```





Revisión (primero desaprobados)